Django использует SQLite по умолчанию с первого дня, и ничего дополнительно устанавливать не нужно: движок встроен в Python, а база — это обычный файл `db.sqlite3` в папке `backend/`. Ниже — гайд, как проверить настройки, создать модель `Message` и написать две функции: вставка и получение с пагинацией по 40.

📋 Таска: SQLite, модель `Message` и функции работы с БД

🎯 Цель

Убедиться, что проект использует SQLite, создать таблицу `messages` с полями `object`, `text`, `date` (и служебным `id`), применить миграции и написать две функции: `create_message()` для вставки и `get_messages()` для получения списка с пагинацией строго по 40 сообщений.

🧠 Что важно знать про SQLite в Django

- **SQLite уже настроен** в `settings.py` при `startproject`. Если вы не меняли `DATABASES`, он там.
- **База — это файл** `db.sqlite3` в корне `backend/` (рядом с `manage.py`). Он создаётся автоматически при первом `migrate`.
- **Файл `db.sqlite3` не коммитится** — он в `.gitignore`. У каждого разработчика своя локальная база, схема воспроизводится миграциями. Проверь, что он в gitignore.
- **SQLite отлично подходит для разработки и MVP**. Для продакшена с высокой нагрузкой позже переедем на PostgreSQL, но сейчас это не задача.

🛠 Шаг 1: Проверить настройки `DATABASES`

Открой `backend/config/settings.py`, найди блок `DATABASES`. Он должен выглядеть так (это дефолт Django):

```
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.sqlite3",
        "NAME": BASE_DIR / "db.sqlite3",
    }
}
```

- `ENGINE` — `django.db.backends.sqlite3` — тот самый встроенный движок.
- `NAME` — `BASE_DIR / "db.sqlite3"` — путь к файлу базы. `BASE_DIR` — это папка `backend/`, значит файл будет `backend/db.sqlite3`.

Если блок отличается — приведи к этому виду. Если `DATABASES` вообще нет — добавь. Больше ничего настраивать не нужно.

> 💡 **Не меняй `NAME` на `:memory:`** — это база в оперативной памяти, которая исчезает при перезапуске сервера. Для разработки нужен файл.

🛠 Шаг 2: Создать приложение (если ещё нет подходящего)

У вас уже есть приложение `crackerchat` с моделью `Post`. Для `Message` можно:

- **Вариант А (проще):** добавить модель `Message` в существующее приложение `crackerchat`.
- **Вариант Б (чище):** создать отдельное приложение `messaging`, если `Message` — это отдельная сущность (например, личные сообщения, а `Post` — общая лента).

В гайде ниже предполагаю **Вариант Б** — отдельное приложение `messaging`. Если хотите в `crackerchat` — просто создайте файлы в `backend/crackerchat/` и не выполняйте `startapp`.

```
python manage.py startapp messaging
```

Добавь в `INSTALLED_APPS` в `settings.py`:

```
INSTALLED_APPS = [
    # ...
    "messaging",
]
```

🛠 Шаг 3: Модель `Message`

Открой `backend/messaging/models.py` и замени содержимое:

```
from django.db import models


class Message(models.Model):
    object = models.CharField(max_length=255)
    text = models.TextField()
    date = models.DateTimeField(auto_now_add=True, db_index=True)

    class Meta:
        ordering = ["-date"]  # новые сверху
        verbose_name = "Сообщение"
        verbose_name_plural = "Сообщения"

    def __str__(self):
        return f"#{self.pk} {self.object}"
```

Что здесь и почему:

- **`object = models.CharField(max_length=255)`** — короткая строка (тема/заголовок). `CharField` обязателен с `max_length`.

  > ⚠️ **Имя `object` — не лучшая практика в Python.** `object` — встроенный тип, и поле `message.object` будет его затенять. Если можно — назови поле `subject` или `title`. Если требование жёсткое («поле должно называться `object`») — оставь как есть, Django это позволяет, но предупреди студентов.

- **`text = models.TextField()`** — длинный текст без ограничения длины на уровне БД.
- **`date = models.DateTimeField(auto_now_add=True, db_index=True)`** — дата и время создания. `auto_now_add=True` проставляет текущий момент **при создании** и не даёт изменить поле вручную. `db_index=True` — индекс по дате, потому что сортировка и пагинация будут именно по ней.
- **`Meta.ordering = ["-date"]`** — дефолтная сортировка «новые сверху». Любой `Message.objects.all()` уже отсортирован, без `order_by` в каждом запросе.
- **`id`** — Django добавляет автоматически как `BigAutoField` (первичный ключ). Отдельно объявлять не нужно.
- **`__str__`** — удобное отображение в админке и shell.

Регистрация в админке (по желанию, но полезно для отладки):

`backend/messaging/admin.py`:

```
from django.contrib import admin
from .models import Message


@admin.register(Message)
class MessageAdmin(admin.ModelAdmin):
    list_display = ("id", "object", "short_text", "date")
    ordering = ("-date",)
    search_fields = ("object", "text")

    @admin.display(description="Текст")
    def short_text(self, obj):
        return obj.text[:80]
```

🛠 Шаг 4: Миграции — создать таблицу `messages`

Из папки `backend/`:

```
python manage.py makemigrations messaging
python manage.py migrate
```

Что происходит:

1. `makemigrations` смотрит на модели в приложении `messaging`, сравнивает с текущим состоянием миграций и генерирует файл `messaging/migrations/0001_initial.py` с инструкцией `CreateModel`.
2. `migrate` применяет эту инструкцию к базе — в `db.sqlite3` появляется таблица `messaging_message` (Django по умолчанию называет таблицы `<app_label>_<model_name_lowercase>`).

> 💡 **Почему таблица называется `messaging_message`, а не просто `messages`?** Django по умолчанию использует префикс приложения, чтобы избежать коллизий между приложениями. Если нужно **строго** имя таблицы `messages` — добавь в `Meta`:
>
> ```
> class Meta:
>     db_table = "messages"
> ```
>
> Но это редко нужно и может усложнить будущие миграции. Рекомендую оставить дефолт.

Проверка, что таблица создана:

```
python manage.py dbshell
```

В открывшейся консоли SQLite:

```
.tables
```

Должна быть строка `messaging_message` (или `messages`, если задал `db_table`). Выйти: `.quit`.

🛠 Шаг 5: Функции для работы с БД

Создай файл `backend/messaging/services.py` — это **сервисный слой**, место для функций, которые работают с моделями. Такой подход отделяет бизнес-логику от views и сериализаторов и упрощает тестирование.

```
from django.core.paginator import Paginator, Page
from django.db import transaction

from .models import Message


@transaction.atomic
def create_message(*, object: str, text: str) -> Message:
    """
    Создаёт новое сообщение и возвращает его.
    Параметры только именованные (keyword-only) — защита от случайной
    передачи аргументов в неправильном порядке.
    """
    object = object.strip()
```

```
    text = text.strip()

    if not object:
        raise ValueError("Поле 'object' не может быть пустым.")
    if not text:
        raise ValueError("Поле 'text' не может быть пустым.")

    return Message.objects.create(object=object, text=text)


def get_messages(*, page: int = 1, per_page: int = 40) -> Page:
    """
    Возвращает страницу сообщений (новые сверху).
    Строго по 40 сообщений на страницу по умолчанию.
    """
    queryset = Message.objects.all()  # ordering уже '-date' из Meta
    paginator = Paginator(queryset, per_page)
    return paginator.get_page(page)
```

Разбор:

- **`@transaction.atomic`** на `create_message` — оборачивает функцию в транзакцию. Для одной вставки это не критично, но закладывает привычку: если внутри функции будет несколько операций записи, они выполнятся атомарно (все или ничего).
- **Keyword-only аргументы (`*`)** — `create_message(object="...", text="...")` нельзя случайно вызвать как `create_message("текст", "объект")` . Это защита от ошибок порядка аргументов.
- **`strip()` + проверка на пустоту** — даже если валидация есть на уровне API, сервисная функция должна быть самодостаточной. Её могут вызвать из shell, из management-команды, из теста.
- **`Message.objects.create(...)`** — сразу сохраняет в БД и возвращает объект с заполненным `id` и `date` .
- **`Paginator(queryset, per_page)`** — стандартный пагинатор Django. Он сам разбивает queryset на страницы.
- **`paginator.get_page(page)`** — безопасный метод: если `page` не число, отрицательное или больше количества страниц, он вернёт **первую** (или последнюю) страницу вместо исключения. Это то, что нужно для API.
- **`Page` объект** содержит:
  - `.object_list` — список сообщений на странице;
  - `.number` — номер текущей страницы;
  - `.paginator.num_pages` — всего страниц;
  - `.has_next()` , `.has_previous()` — есть ли следующая/предыдущая;
  - `.next_page_number()` , `.previous_page_number()` — номера соседних страниц.

Строго 40: `per_page` по умолчанию `40` . Если нужно жёстко зафиксировать — убери параметр `per_page` и передавай `40` в `Paginator` явно:

```
paginator = Paginator(queryset, 40)
```

Но оставить параметр полезно для тестов (например, `get_messages(page=1, per_page=5)`).

🛠 Шаг 6: Проверка через Django shell

```
python manage.py shell
```

```
from messaging.services import create_message, get_messages

# Создать сообщение
msg = create_message(object="Привет", text="Это тестовое сообщение")
print(msg.id, msg.object, msg.date)

# Получить страницу
page = get_messages(page=1)
print("Всего:", page.paginator.count)
print("Страниц:", page.paginator.num_pages)
print("На странице:", len(page.object_list))
print("Есть следующая:", page.has_next())

# Создать 50 сообщений для проверки пагинации
for i in range(50):
    create_message(object=f"Тема {i}", text=f"Текст {i}")

page1 = get_messages(page=1)
page2 = get_messages(page=2)
print("Стр.1:", len(page1.object_list), "Стр.2:", len(page2.object_list))
print("Стр.1 has_next:", page1.has_next())
print("Стр.2 has_next:", page2.has_next())
```

Ожидаемый результат: на первой странице 40 сообщений, на второй — остаток (11, если создали 1 + 50), `page1.has_next() == True`, `page2.has_next() == False` (если всего 51). Порядок — новые сверху (последнее созданное сообщение первым).

Выйти из shell: `exit()`.

🚫 Подводные камни

- **`no such table: messaging_message`** — не выполнен `migrate` после `makemigrations`. Проверь: `python manage.py showmigrations messaging`.
- **`django.db.utils.OperationalError: no such column`** — модель изменили, но не сделали `makemigrations` + `migrate`. Всегда: изменил модель → `makemigrations` → `migrate`.
- **`object` как имя поля** — Django не упадёт, но в Python `message.object` затеняет встроенный `object`. Если кодстайл-линтеры ругаются — переименуй в `subject`. Если оставляешь —

предупреди студентов.

- **SQLite и конкурентная запись** — SQLite блокирует базу на запись при одновременных вставках из нескольких процессов. Для нашей нагрузки (анонимная борда с одним dev-сервером) это не проблема. Для продакшена с несколькими воркерами позже переедем на PostgreSQL.
- **`db.sqlite3` в git** — не должен попадать. Он в `.gitignore`. Если случайно закоммитил — удали из индекса: `git rm --cached backend/db.sqlite3`.
- **`Paginator` на огромной таблице** — `count()` на миллионах строк может быть медленным. Для нашего масштаба (тысячи сообщений) это неактуально. Если станет проблемой — заменим на cursor-based пагинацию позже.

📝 Критерии готовности (Definition of Done)

- [ ] В `settings.py` блок `DATABASES` указывает на `django.db.backends.sqlite3` и файл `BASE_DIR / "db.sqlite3"`.
- [ ] Приложение `messaging` создано и добавлено в `INSTALLED_APPS`.
- [ ] Модель `Message` с полями `object` (`CharField`), `text` (`TextField`), `date` (`DateTimeField(auto_now_add=True, db_index=True)`).
- [ ] `Meta.ordering = ["-date"]`.
- [ ] Миграция `0001_initial` сгенерирована и применена; таблица существует в `db.sqlite3`.
- [ ] `messaging/services.py` содержит `create_message()` (с `@transaction.atomic` и проверкой на пустоту) и `get_messages()` (с `Paginator` и `per_page=40`).
- [ ] В shell: `create_message()` создаёт запись, `get_messages()` возвращает первую страницу; после создания 50+ сообщений на первой странице ровно 40, на второй — остаток, `has_next()` работает корректно.
- [ ] Изменения закоммичены (без `db.sqlite3`).